



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09

NOMBRE COMPLETO: Pablo Arturo Gonzalez Cuellar

N° de Cuenta: 319241013

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 04

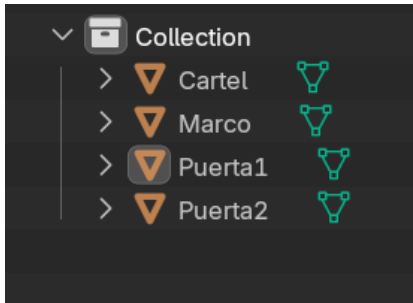
SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 02/11/2025

CALIFICACIÓN: _____

1.- Separar del arco la parte del letrero y las puertas o rejas

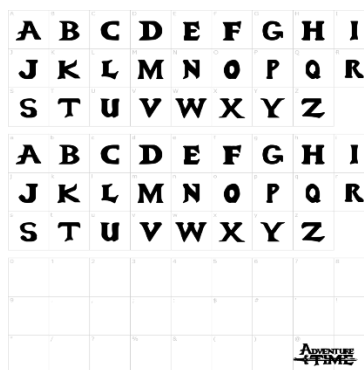
2.- Hacer que en el arco que crearon se muestre la palabra: **PROYECTO CGEIH** desplazándose las letras de derecha a izquierda como si fuera letrero LCD/LED de forma cíclica



Para esta primer parte de la práctica se exportaron los modelos por separado de nuestro modelo original, y luego las importamos a OpenGL

```
Marco_M = Model();
Marco_M.LoadModel("Models/Marco.fbx");
Letrero_M = Model();
Letrero_M.LoadModel("Models/Letrero.fbx");
PuertaDerecha_M = Model();
PuertaDerecha_M.LoadModel("Models/PuertaDerecha.fbx");
PuertaIzquierda_M = Model();
PuertaIzquierda_M.LoadModel("Models/PuertaIzquierda.fbx");
Dado_M = Model();
Dado_M.LoadModel("Models/Dado.fbx");
```

Para crear el efecto de nuestro letrero tipo LED primero agregamos nuestra textura de las letras, en este caso usamos la siguiente imagen:



Cargamos la textura

```
AdvTime_FontTexture = Texture("Textures/AdvTime_Font.png");
AdvTime_FontTexture.LoadTextureA();
```

Definimos un vector `offsets` que guardará las coordenadas de textura U y V de cada letra para formar la frase "PROYECTO CGEIH C"

```
std::vector<glm::vec2> offsets = {
    glm::vec2(0.65f, 0.57f), // P
    glm::vec2(0.88f, 0.57f), // R
    glm::vec2(0.57f, 0.56f), // O
    glm::vec2(0.651f, 0.46f), // Y
    glm::vec2(0.45f, 0.67f), // E
    glm::vec2(0.23f, 0.67f), // C
    glm::vec2(0.11f, 0.46f), // T
    glm::vec2(0.57f, 0.56f), // O
    glm::vec2(0.23f, 0.67f), // C
    glm::vec2(0.655f, 0.67f), // G
    glm::vec2(0.45f, 0.67f), // E
    glm::vec2(0.86f, 0.67f), // I
    glm::vec2(0.77f, 0.67f), // H
    glm::vec2(0.23f, 0.67f) // C
};
```

Después, creamos variables para controlar tanto la animación como el formato de nuestras letras

```
static float tiempoAcumulado = 0.0f;
static int indiceInicio = 0;
int letrasVisibles = 4;
float intervaloCambio = 40.0f;
glm::vec3 basePos(-12.0f, 2.5f, 9.5f);
float separacion = 1.5f;
glm::vec3 escalaLetras(1.5f, 2.0f, 2.0f);
```

Dentro del `while` agregamos la lógica de la animación. La secuencia es, una vez que nuestra variable `tiempoAcomulado` supera nuestro `intervaloCambio` movemos el índice, el cual controlará qué letra debe aparecer al principio del letrero.

```

tiempoAcumulado += deltaTime;
if (tiempoAcumulado >= intervaloCambio) {
    tiempoAcumulado = 0.0f;
    indiceInicio++;
    if (indiceInicio > (int)offsets.size() - letrasVisibles)
        indiceInicio = 0;
}

```

Finalmente, renderizamos por medio de un ciclo for, donde en cada iteración calculamos el índice de la letra que se mostrará y le pasamos las coordenadas UV al shader. También calculamos su posición con `letraModel` para que siempre quede una junto a la otra

```

for (int i = 0; i < letrasVisibles; i++) {
    int idx = (indiceInicio + i) % offsets.size();

    glm::vec2 toffset = offsets[idx];
    glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));

    glm::mat4 letraModel = glm::mat4(1.0f);
    letraModel = glm::rotate(letraModel, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
    float startX = basePos.x + (letrasVisibles - 1) * separacion;
    letraModel = glm::translate(letraModel,
        glm::vec3(startX - i * separacion, basePos.y, basePos.z));

    letraModel = glm::scale(letraModel, escalaLetras);
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(letraModel));

    meshList[4]->RenderMesh();
}

```

3.- Hacer que al presionar una tecla una de las puertas se abra por medio de una rotación y la otra puerta se abra por medio de un deslizamiento.

Una vez importados los modelos y modificarles previamente su punto pivote en Blender, manejamos por separado la lógica del movimiento de la puerta que rotará y la puerta que se deslizará.

Primero, para la rotación de una de las puertas, asignamos la tecla O para el movimiento.

```

// Abrir/cerrar con tecla O
if (mainWindow.getKeys()[GLFW_KEY_O]) {
    if (!teclaOPresionada) {
        abierta = !abierta;
        teclaOPresionada = true;
    }
}
else {
    teclaOPresionada = false;
}

```

Ahora, definimos la lógica del movimiento donde decimos que si el movimiento está a -90 grados se considere *abierta* y por el contrario si es 0 se considere *cerrada*.

```
// Animación
float targetAng = abierta ? -90.0f : 0.0f;
if (fabs(ang - targetAng) > 0.1f) {

    float step = vel * deltaTime/10;
    if (ang < targetAng) ang = std::min(ang + step, targetAng);
    else ang = std::max(ang - step, targetAng);
}
```

Por último, al renderizar el modelo, solo le pasamos esta variable *ang* a nuestro *rotate*

```
model = modelaux;
model = glm::translate(model, glm::vec3(-3.3f, 0.0f, 4.7f));
model = glm::rotate(model, -ang * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(glm::vec2(0.0f)));
Puerta.UseTexture();
PuertaIzquierda_M.RenderModel();
```

Para la parte de la puerta que se desliza se requería realizar dos animaciones, una la cual “sacara” la puerta en el eje Z y luego en X para simular el deslizamiento.

Primero asignamos la letra P para este movimiento, así como también establecemos la fase inicial, 1 para abrir y 3 para cerrar

```
// Animación de Puerta Derecha
if (mainWindow.getKeys()[GLFW_KEY_P]) {
    if (!teclaPPresionada && !puertaDerechaAnimando) {
        puertaDerechaAnimando = true;
        teclaPPresionada = true;
        if (!puertaDerechaAbierta) {
            fase = 1;
        }
        else {
            fase = 3;
        }
        progresoAnim = 0.0f;
    }
}
else {
    teclaPPresionada = false;
}
```

Para la lógica del movimiento decimos que si la animación está activa se actualice la posición *puertaDerechaPos* basándose en la fase actual. Una vez que la fase se completa, se pasa a la siguiente.

```
// Lógica de movimiento de la puerta
if (puertaDerechaAnimando) {
    progresoAnim += deltaTime/20;
    float t = progresoAnim / duracionFase;
    if (fase == 1) {
        puertaDerechaPos.z = glm::min(t, 1.0f) * 0.5f;
        if (t >= 1.0f) { fase = 2; progresoAnim = 0.0f; }
    }
    else if (fase == 2) {
        puertaDerechaPos = glm::vec3(glm::min(t, 1.0f) * 4.0f, 0.0f, 1.0f);
        if (t >= 1.0f) {
            puertaDerechaAnimando = false;
            puertaDerechaAbierta = true;
            fase = 0;
        }
    }
    else if (fase == 3) {
        puertaDerechaPos.z = glm::vec3((1.0f - glm::min(t, 1.0f)) * 4.0f, 0.0f, 1.0f);
        if (t >= 1.0f) { fase = 4; progresoAnim = 0.0f; }
    }
    else if (fase == 4) {
        puertaDerechaPos.z = (1.0f - glm::min(t, 1.0f)) * 0.5f;
        if (t >= 1.0f) {
            puertaDerechaAnimando = false;
            puertaDerechaAbierta = false;
            fase = 0;
        }
    }
}
}
```

Finalmente al renderizar, la posición calculada `puertaDerechaPos` se aplica a nuestro `translate`

```
model = modelaux;
model = glm::translate(model, glm::vec3(puertaDerechaPos.x, puertaDerechaPos.z, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(glm::vec2(0.0f)));
Puerta.UseTexture();
PuertaDerecha_M.RenderModel();
```

Extra 5 puntos: Su dado de 8 caras cae girando sobre el piso, y muestra un número random siempre hacia arriba, se repite la tirada al presionar una tecla.

Para el ejercicio extra, primero importamos nuestro dado

```
Dado_M = Model();
Dado_M.LoadModel("Models/Dado.fbx");
```

Después, agregamos el control de la animación, el punto más importante de nuestra animación, aquí decimos que al presionar la letra "I", si el dado está en el suelo (con la variable `dadoEnSuelo`) se reinicia a su posición inicial, de lo contrario, si el dado está en su posición inicial, comienza la caída, en ese momento se genera un número aleatorio para la cara `caraDado` y una velocidad de rotación

```
// Animacion del Dado
if (mainWindow.getKeys()[GLFW_KEY_I]) {
    if (!teclaIPresionada) {
        teclaIPresionada = true;
        if (!dadoAnimando && !dadoEnSuelo) {
            dadoAnimando = true;
            velDado = glm::vec3(0.0f, 0.0f, 0.0f);
            rotDadoActual = glm::vec3(0.0f);
            rotVelocidad = glm::vec3(
                180.0f + rand() % 90,
                180.0f + rand() % 90,
                180.0f + rand() % 90
            );
            caraDado = rand() % 9;
        }
        else if (dadoEnSuelo) {
            dadoEnSuelo = false;
            dadoAnimando = false;
            posDado = glm::vec3(5.0f, 10.0f, -5.0f);
            velDado = glm::vec3(0.0f);
            rotDadoActual = glm::vec3(0.0f);
        }
    }
    else {
        teclaIPresionada = false;
    }
}
```

Después, mientras nuestra variable `dadoAnimando` sea verdadera, se aplica la gravedad a la velocidad en Y, se actualiza la posición de nuestro dado y se le aplica la rotación dinámica. Una vez que se hace todo este proceso, la simulación se detiene al detectar la colisión con el suelo, activando de esta manera la bandera `dadoEnSuelo`

```
// Actualizacion del movimiento y rotacion
if (dadoAnimando) {
    // aplicar gravedad
    velDado.y += gravedad * deltaTime;
    posDado += velDado * deltaTime;

    // rotación continua en tres ejes
    rotDadoActual += rotVelocidad * deltaTime;

    // detener al tocar el suelo
    if (posDado.y <= 0.5f) {
        posDado.y = 0.5f;
        dadoAnimando = false;
        dadoEnSuelo = true;
    }
}
```

Finalmente al renderizar nuestro dado, vamos a tener diversos escenarios, si el dado está cayendo, se aplica la rotación dinámica `rotDadoActual`. De lo contrario, si ya está en el suelo usamos un switch en el que, dependiendo del valor de nuestra variable `caraDado` aplica una rotación fija específica para que esa cara quede viendo hacia arriba.

```

//dado
model = glm::mat4(1.0);
model = glm::translate(model, posDado);
model = glm::rotate(model, glm::radians(40.0f), glm::vec3(1, 0, 0));
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0, 0, 1));

if (dadoAnimando) {
    model = glm::rotate(model, glm::radians(rotDadoActual.x), glm::vec3(1, 0, 0));
    model = glm::rotate(model, glm::radians(rotDadoActual.y), glm::vec3(0, 1, 0));
    model = glm::rotate(model, glm::radians(rotDadoActual.z), glm::vec3(0, 0, 1));
}

else if (dadoEnSuelo) {
    switch (caraDado) {
        case 0: break;
        case 1: model = glm::rotate(model, glm::radians(80.0f), glm::vec3(0, 1, 0));
            model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0, 0, 1));
            break;
        case 2: model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0, 1, 0)); break;
        case 3: model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0, 1, 0));
            model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0, 0, 1));
            break;
        case 4: model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0, 1, 0));
            model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0, 0, 1));
            break;
        case 5: model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0, 0, 1)); break;
        case 6: model = glm::rotate(model, glm::radians(360.0f), glm::vec3(0, 1, 0)); break;
        case 7: model = glm::rotate(model, glm::radians(270.0f), glm::vec3(0, 1, 0)); break;
        case 8: model = glm::rotate(model, glm::radians(270.0f), glm::vec3(0, 1, 0));
            model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0, 0, 1));
            break;
    }
}
}

```

CAPTURAS DE EJECUCIÓN:

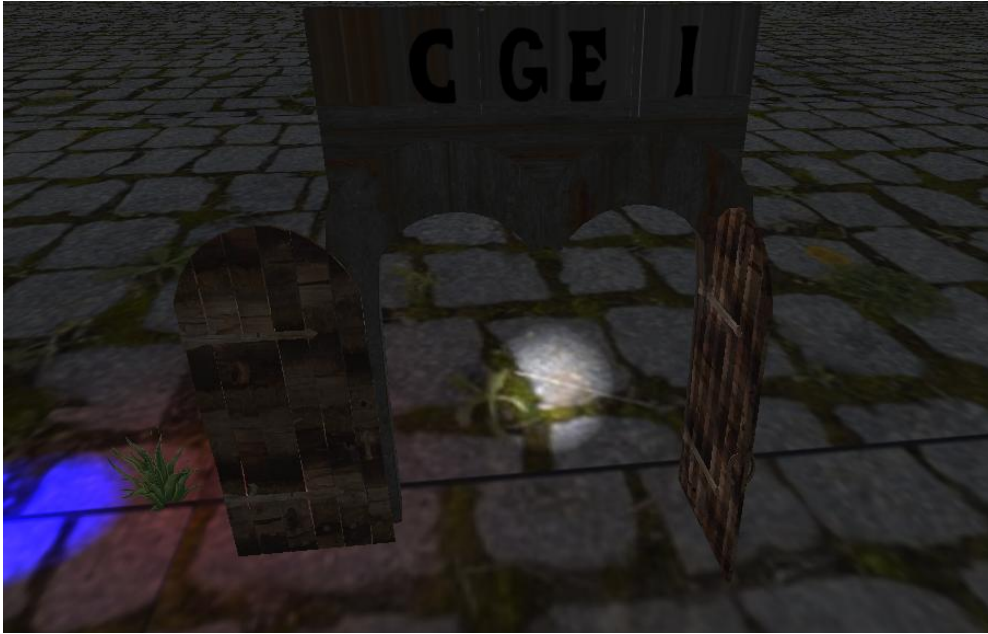
EJERCICIO 2.-





EJERCICIO 3.-





EJERCICIO EXTRA.-





PROBLEMAS SURGIDOS:

El mayor reto de esta práctica fue ajustar las rotaciones finales en el switch del dado ya que fue un proceso de prueba y error. Se tuvo que ir probando qué combinación de rotaciones en X, Y y Z lograba que cada una de las 8 caras (más la base) quedara viendo perfectamente hacia arriba.

CONCLUSIÓN

En esta práctica se reforzaron conceptos muy importantes en la animación, mostrando la evolución del camino en esta materia de computación Gráfica, ya que, partimos desde escenas estáticas con modelos primitivos, hasta escenas dinámicas que responden acorde a una lógica implementada en OpenGL como lo fue la manipulación de las coordenadas de textura para simular el letrero LED o la animación del dado. Todo esto en conjunto y al desarrollar esta práctica, muestra que la animación no es solo una técnica que se puede solo implementar y ya, sino que un conjunto de técnicas y estrategias que deben seleccionarse, combinarse con lógica de programación y adaptarse al resultado que se requiere.